PYTHON PROGRAMMING / WEEK 12 / SESSION 2

Validation, Feature Tests and Code Review

Extend one feature while keeping the MVP reliable.

120 minutes, including a break / Instructor: MinGi KYUNG

ENGLISH EDITION

Today’s Deliverable

- Complete one feature and tests that demonstrate its behavior.
- Check that expected failures preserve the previous report.
- Work in your own repository or a copy of the example.

Baseline: Existing Tests

First confirm that all 12 existing Week 11 MVP tests pass.

```
cd 11week/examples/korea_market_mvp
python3 -m unittest -v
```

Activity A: Predict First

Activity A takes 5 minutes in total. Predict individually first.

Direct function arguments	Accept or reject?
0, 100, -1	Write your reason
True, "100", 1.5	Write your reason

Activity A: Run and Compare

Run from the repository root.

```
python3 12week/examples/validation_lab/playground.py verdict
```

- Choose one value you and your partner classified differently.
- Explain the verdict using the function contract.

The CSV Input Boundary

- CSV cells arrive as strings.
- Check the schema and required fields first.
- Pass validated, converted values to the analysis function.

Stock Codes Stay Text

```
int("005930")  # 5930
```

- Leading zeros are part of a stock code.
- Convert prices and volumes used in calculations to integers.

Conversion and Valid Values

```
int("-1")  # -1
```

- Successful conversion does not make negative volume valid.
- Prices must be positive. Volume must be nonnegative.

Row Rules and Snapshot Rules

Scope	Rule
One row	Real date, valid price and volume
Whole input	One date, unique stock codes

Include whitespace normalization in the input contract.

The Minimum-Volume Type Contract

- Accept only nonnegative int values.
- Reject bool, strings and floats.
- Distinguish direct function calls from CLI string conversion.

Code That Rejects Invalid Input

```
def validate_min_volume(value):
    if type(value) is not int or value < 0:
        raise ValueError("min_volume must be a nonnegative integer")
    return value
```

isinstance(True, int) is True, so account for bool explicitly.

Where Exceptions Originate

- Rule functions raise ValueError for contract violations.
- Storage propagates file and decoding failures to its caller.

Where Exceptions Are Handled

- The CLI converts expected failures into a message and exit code.
- Handle exceptions where the program can make a useful decision.
- Keep unexpected programming errors visible during development.

Failure Disguised as Empty Data

```
try:
    quotes = load_quotes(path)
except Exception:
    quotes = []
```

Missing files and programming errors both look like valid empty input.

Failure and Valid Empty Results

Input	Expected behavior
Missing CSV	Fail, preserve the old report
Valid header-only CSV	Succeed, generate an empty report

A zero-byte file is different from a header-only file.

The Existing CLI Handler

```
except (OSError, ValueError, UnicodeError) as error:
    print(f"Report not generated: {error}")
    return 1
```

Include diagnostic context such as the CSV row number.

Exit Codes Are Observable Results

Code	Meaning
0	Report generated successfully
1	Expected data or file failure
2	argparse argument error

Check the output file as well as the message.

Activity B: Predict CSV Outcomes

Activity B takes 10 minutes in total. Predict before running.

- Cases: valid, header-only, missing file and negative volume.
- Record the return code and whether the OLD report survives.

Activity B: Run and Add a Case

```
python3 12week/examples/validation_lab/playground.py csv
```

- Add an invalid-date or missing-column case to cases.
- Change one condition at a time. Compare prediction with observation.

Activity B: Explain the Observation

- Why does a valid empty result replace OLD?
- Why should an input error preserve OLD?
- Record the evidence and one sentence explaining each result.

Before Replacing the Report

- Read and validate all input.
- Write the complete new HTML to a temporary file in the output folder.
- Keep the old report until validation and writing finish.

Replacement and Cleanup

- Replace the destination with os.replace after writing finishes.
- Remove the temporary file if replacement fails.

Scope: protect the old result on handled pre-replacement failures. This does not guarantee power-loss durability.

Unit Tests: Calculation Rules

```
self.assertAlmostEqual(change_pct(100, 110), 10.0)
```

- Check the contract of one small function.
- A failure narrows the area to investigate.

Integration and User Acceptance

Check	Observable result
Call main with malformed CSV	Failure code, old HTML preserved
Open the report in a browser	Market, labels and empty state

Check both function behavior and the user’s workflow.

Isolation with Temporary Files

```
from pathlib import Path
from tempfile import TemporaryDirectory

with TemporaryDirectory() as folder:
    output = Path(folder) / "index.html"
    output.write_text("OLD", encoding="utf-8")
```

Use this disposable path for failure experiments.

Reproducing a Rare Write Failure

```
with patch("storage.os.replace", side_effect=OSError("disk")):
    write_html(output, "NEW")
```

- Patch storage.os.replace, the name used by the tested module.
- Use real temporary files for CSV parsing experiments.

Activity C: Predict the Rescue

Activity C takes 8 minutes in total. Predict replacement failure.

- Will OSError reach the caller?
- Will the output contain OLD or NEW?
- Will temporary files remain in the folder?

Activity C: Turn Observations into Assertions

```
python3 12week/examples/validation_lab/playground.py rescue
```

- Check propagation with assertRaises.
- Check OLD and the remaining files with assertEqual.
- Complete and run the test_rescue.py worksheet skeleton.

New Feature: Minimum Volume

User goal: compare stocks whose volume meets a chosen minimum.

```
analyze(quotes, market, min_volume=0)
```

The default 0 preserves existing two-argument calls and results.

Validation Before Filtering

- Reject duplicate codes and mixed dates across the whole input first.
- Apply market and minimum-volume conditions together.
- Preserve change-descending, code-ascending order and the input data.

Acceptance: The Equality Boundary

Volume	With threshold 100
99	Exclude
100	Include
101	Include

Derive expectations from the contract, not the implementation.

Acceptance: Errors and Empty Results

Input	Expected result
-1, True, "100", 1.5	ValueError for each
All stocks excluded	[], summary count=0, mean=None
Duplicate hidden by filtering	ValueError

Activity D: Faults and Minimal Inputs

Activity D takes 8 minutes in total. Find a minimal input for each fault.

- A compares volume > threshold.
- B checks types only with isinstance(value, int).
- C checks duplicate codes after filtering.

Activity D: Exchange Inputs

- Hide the candidate names. Give your partner inputs and expectations.
- Your partner explains which faults those inputs expose.
- Add the equality boundary and hidden duplicate to your test plan.

Break — 10 Minutes

Choose one feature to extend in your own project.

After the break, start with its contract and a failing test.

Feature Contract — Start of the 10-Minute Block

- Open labs/session2/project-review.md.
- Link a requirement ID to the function you will change.
- State accepted inputs, successful results and errors.

Failure Evidence Before Implementation

- Add a test for the new behavior first.
- Run it on the old code and check that it fails for the expected reason.
- Record the input, expectation and actual failure.

Implementation — 20-Minute Block

- Validate the new parameter before applying filters.
- Keep whole-snapshot validation before filtering.
- Leave the input unchanged.

Feature and Regression Checks

- Run the new cases and the complete existing suite.
- Check default 0 and existing two-argument calls.
- For a failure, reproduce the input and narrow down its cause.

Demo Preparation — 10-Minute Block

- Generate normal and empty reports with the existing main entry point.
- Prepare a repeatable test that demonstrates the new function.
- Without CLI wiring, demonstrate the report and function tests separately.

Activity E: A 90-Second User Demo

- 30 seconds: the user identifies the market, count and ranking.
- 30 seconds: the user evaluates the empty-state message.
- 30 seconds: the author runs the inclusive-threshold test.

Recording Demo Observations

- Record one observed issue or one confirmed acceptance criterion.
- Link it to the demonstration field in project-review.md.
- Ensure the final submitted suite passes.

Code Review: Find a Counterexample — 15 Minutes

- Compare the diff with the contract and test plan.
- Seek a counterexample among boundaries, errors and empty results.
- For solo work, review your own diff using the same questions.

Code Review: Concrete Feedback

```
Input: ______
Expected / actual result: ______ / ______
Supporting function or test: ______
```

Also check failure propagation, old-output preservation and responsibilities.

Revision and Rechecking

- Revise the code and rerun the affected tests.
- Record a reason for feedback you do not apply.
- Save the feedback and revision result in the review sheet.

Evidence for the PR

- State the problem, user-visible behavior and requirement ID.
- Name the changed function, exact checks and actual results.
- If GitHub is unavailable, submit a local diff and review record.

Submission and Exit Questions — 5 Minutes

- Submit the feature, tests, review record and traceability documents.
- What specific fault does your test catch?
- Why must filtering never hide a duplicate code?

Continuing into Week 13

- Make one final check that the original MVP still runs.
- Next week, extend data processing or UI on top of these checked rules.